



FACULTY OF ENGINEERING

UNIVERSITY OF RUHUNA

DEPARTMENT OF COMPUTER ENGINEERING

EC7207 - High Performance Computing

Project Analysis Report

Hashed Password Cracking System

NAME : WIJERATHNA W.M.M.T

REG NO : EG/2020/4291

SEMESTER : 07

DATE : 18/07/2025

Abstract

This project develops and analyzes a high-performance password cracker using different technologies to recover passwords from their SHA-256 hashed forms through brute-force techniques. The system works by generating all possible combinations within a specified length range and character set, computing their SHA-256 hashes, and comparing them against the target hash to find the original password. Special attention is given to optimizing computational efficiency since brute-force attacks require checking a large number of possibilities.

To achieve maximum performance, the project uses parallel, distributed, and hybrid computing approaches. For parallel computing on a single machine with multiple cores, OpenMP is used. It allows easy implementation of loop-level parallelism through simple pragma directives, making thread management automatic and improving speed. For handling larger workloads, MPI (Message Passing Interface) enables distributed computing across multiple machines in a cluster, significantly reducing processing time by dividing the task.

1. Project Overview

1.1. Explanation about tech stack

The project implements two parallel computing approaches for password cracking. For shared-memory systems, OpenMP was selected over POSIX threads due to its simpler configuration and automatic thread management capabilities. For distributed computing, MPI was chosen instead of CUDA to ensure broader compatibility, allowing the solution to run effectively even on low-performance computers without specialized GPU hardware. This makes the project more generalized and accessible across different computing environments.

1.2. Password Vulnerability Analysis

Passwords become vulnerable primarily through three weaknesses. First, short length passwords are easily cracked as they have limited combinations. Second, passwords using limited character sets (such as numeric-only or lowercase-letters-only) dramatically reduce the search space. Third, even complex passwords can be weak if they follow predictable patterns - for instance, 'Name@123' combines uppercase, lowercase, numbers and special characters but remains guessable due to common naming conventions. The project specifically examines Alpha- numeric-only passwords within maximum four characters as they represent a common weak password pattern.

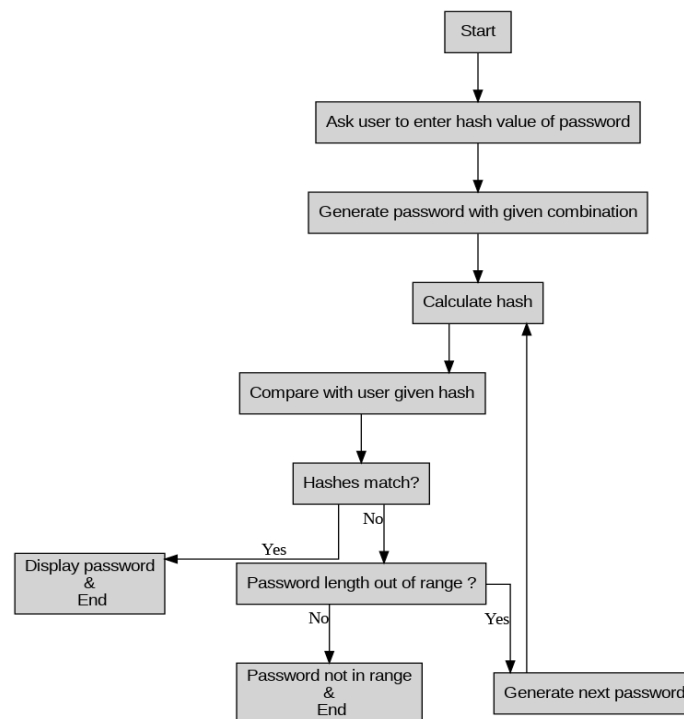
1.3. Worst case

The system tests against the full alphanumeric character set containing 62 characters (abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789). The worst-case computational challenge occurs with maximum-length passwords (9 characters in this combination) where the program must exhaustively check all possible combinations from single-character (62 possibilities) up to 9-character passwords. This exponential growth in combinations demonstrates why longer, complex passwords provide better security, as the time required to crack them increases with each additional character.

2. Architecture

The password cracker is built on WSL (Windows Subsystem for Linux) running over Windows OS. The main programming language used is C because it's fast and good for system-level programming. For compiling the code, we use GCC compiler - it works for both the simple serial version and the parallel versions using OpenMP and MPI. The program uses several important libraries: `stdio.h` for basic input/output operations, `string.h` for handling text/strings, `openssl/sha.h` for calculating SHA-256 hash values (this helps in checking passwords), and `time.h` to measure how long the cracking process takes. These libraries help make the program work properly for all its tasks like generating passwords, calculating hashes, and comparing them.

2.1. Flow of Project



This flowchart explains how the password cracker program works in simple steps. First, the program asks the user to enter the SHA-256 hash value of the password they want to crack. Then it starts generating possible password combinations one by one based on the given character set (like numbers, letters etc.). For each generated password, it calculates its hash value and checks if it matches the user's input hash. If it finds a match, it displays the cracked password and stops. If no match is found and all possible password lengths have been tried (like from 1 character up to maximum length), it tells the user that password couldn't be found in that range. Otherwise, it continues generating and checking the next possible password until either the password is found or all combinations are exhausted. This process keeps running automatically until it gets a result.

2.2. User Interactions

2.2.1. Parameter Configuration

Before starting the cracking process, users need to configure the search parameters. This includes setting the maximum password length, where longer passwords will significantly increase the computation time. Users can also specify the character set to be used in the search, choosing between numeric only, alphanumeric, or including special characters. Broader character selections expand the search space exponentially, resulting in longer processing times

2.2.2. Program Execution

Once the parameters are set, users initiate the password cracking process by running the program. The system immediately begins working through all possible combinations within the defined constraints, employing the selected computing method (serial, OpenMP, or MPI) to optimize the search.

2.2.3. Hash Input Method

The target SHA-256 hash value can be provided through two methods: manual typing or pasting from another source. For testing purposes, users may generate sample hashes using online tools such to verify the system's functionality before attempting to crack actual password hashes.

2.2.4. Search Process

During operation, the program systematically generates potential password combinations, computes their SHA-256 hash values, and compares each result against the target hash. This brute-force approach continues until either a matching hash is found or all possible combinations within the specified parameters have been exhausted.

2.2.5. Result Presentation

Upon completion, the program displays one of two outcomes: either the successfully recovered password along with the total time taken for the operation, or a notification that the password could not be found within the defined search space. The results include performance metrics to help users understand the efficiency of the cracking process.

3. Implementation

3.1. Serial Version

The serial version works on just one CPU core. It checks all possible password combinations one by one. For each password, it calculates the SHA-256 hash and compares it with the target hash. This is the slowest method but simplest to implement. It's good for testing small password ranges but becomes very slow for longer passwords.

3.2. OpenMP Version

The OpenMP version makes it faster by using all CPU cores together. It splits the password combinations between different cores using `#pragma omp parallel for`. Each core checks its assigned passwords independently. When any core finds the correct password, it tells others to stop searching. This version works well on modern computers with multiple CPU cores, giving much better speed than the serial version.

3.3. MPI Version

The MPI version can use multiple computers working together. One computer acts as the master that divides the work and sends parts to other computers (workers). Each worker checks its assigned password range and reports back if it finds a match. This is useful when you have many computers available, as it can handle very large password searches by distributing the work.

3.4. Hybrid Version

The hybrid version combines OpenMP and MPI for maximum performance. Within each computer, OpenMP uses all CPU cores. Between computers, MPI coordinates the work. This gives two levels of parallel processing - using multiple cores on each computer and using multiple computers together. It's the most powerful version for large-scale password cracking tasks.

4. Results and Analysis

4.1. Serial

	Example	Time (s)
Near Best case	aa	0.002
Worst case	9999	33.711

The serial implementation shows significant variation in cracking time depending on password complexity. For simple 2-character passwords like "aa", it takes just 0.002 seconds—demonstrating how vulnerable short passwords are even to basic brute-force attacks. However, the time surges to 33.7 seconds for a 4-digit numeric password ("9999"), highlighting how quickly computational demands grow with password length.

4.2. Open MP

Password Type	Threads	Example	Time (s)
Near Best case	Default	aa	0.018
Worst case	Default	9999	11.632
Worst case	2	9999	24.352
Worst case	4	9999	18.609
Worst case	6	9999	15.105
Worst case	8	9999	13.213
Worst case	10	9999	14.206
Worst case	12	9999	12.958
Worst case	14	9999	13.825
Worst case	16	9999	10.148
Worst case	18	9999	11.185
Worst case	20	9999	12.090

The OpenMP implementation demonstrates how parallel processing significantly improves password cracking speeds compared to the serial version, though with diminishing returns as thread count increases. For the worst-case scenario (4-digit "9999"), the default OpenMP configuration completes in 11.632 seconds - nearly 3× faster than the serial version's 33.7 seconds. Performance scales with additional threads, with 16 threads achieving optimal speed (10.148s), but interestingly, using more than 16 threads (e.g., 20 threads at 12.09s) shows slightly reduced efficiency due to thread management overhead. The near-best case ("aa") at 0.018s remains consistently fast across configurations, confirming that short passwords remain vulnerable regardless of parallelization.

4.3. MPI

Password Type	Np	Example	Time (s)
Near Best case	2	aa	0.007
Worst case	2	9999	21.425
Worst case	4	9999	15.637
Worst case	6	9999	15.364
Worst case	8	9999	11.996
Worst case	10	9999	12.385
Worst case	12	9999	13.347
Worst case	14	9999	9.558
Worst case	16	9999	12.006
Worst case	18	9999	11.533
Worst case	20	9999	13.538

The MPI implementation shows how distributing the workload across multiple processors can speed up password cracking. For simple passwords like "aa", it takes just 0.007 seconds with 2 processors - much faster than both serial and OpenMP versions. For tougher 4-digit passwords ("9999"), the time improves as we add more processors: from 21.4 seconds (2 processors) down to 9.5 seconds (14 processors). However, adding too many processors (like 20) actually makes it slower again (13.5 seconds) because of communication overhead between processors. The best performance comes at 14 processors (9.5 seconds), which is about 3.5 times faster than the serial version. This shows MPI is very effective for password cracking, but only up to a certain number of processors - after that point, the extra coordination between processors starts to slow things down instead of helping. The results prove that distributed computing can handle complex passwords much better than single-computer methods.

4.4. Hybrid

4.4.1. Constant number of threads (8)

Password Type	Np	Example	Time (s)
Worst case	2	9999	16.519
Worst case	4	9999	16.504
Worst case	6	9999	15.767
Worst case	8	9999	14.983
Worst case	10	9999	15.671
Worst case	default	9999	12.886

The hybrid version (combining OpenMP and MPI) shows interesting results when cracking the 4-digit password "9999" with 8 constant threads. While increasing MPI processes from 2 to 8 gives gradual improvement (from 16.5s to 14.9s), the best performance comes with default settings at 12.8 seconds - faster than most configurations. This suggests that simply adding more MPI processes doesn't always help after a certain point, as the communication between different processes starts eating into the performance gains. The hybrid approach works better than pure MPI (which gave 9.5s best case) but doesn't outperform OpenMP's best result (10.1s) for this password length, showing that for medium-complexity passwords, a well-tuned OpenMP implementation on a single multi-core machine might be more efficient than distributed computing.

4.4.2. Constant number of processes (4)

Password Type	Threads	Example	Time (s)
Near Best case	2	aa	0.048
Worst case	2	9999	13.048
Worst case	4	9999	14.668
Worst case	6	9999	16.912
Worst case	8	9999	15.699

The hybrid version with constant 4 MPI processes shows mixed results across different thread counts. For simple passwords like "aa", it cracks quickly in 0.048 seconds with just 2 threads. However, for the tough 4-digit "9999" password, performance varies unexpectedly - the fastest time (13.048s) comes with only 2 threads, while adding more threads actually makes it slower (peaking at 16.9s with 6 threads). This means that for this setup, using fewer threads per MPI process works better than overloading each process with many threads.

5. Accuracy

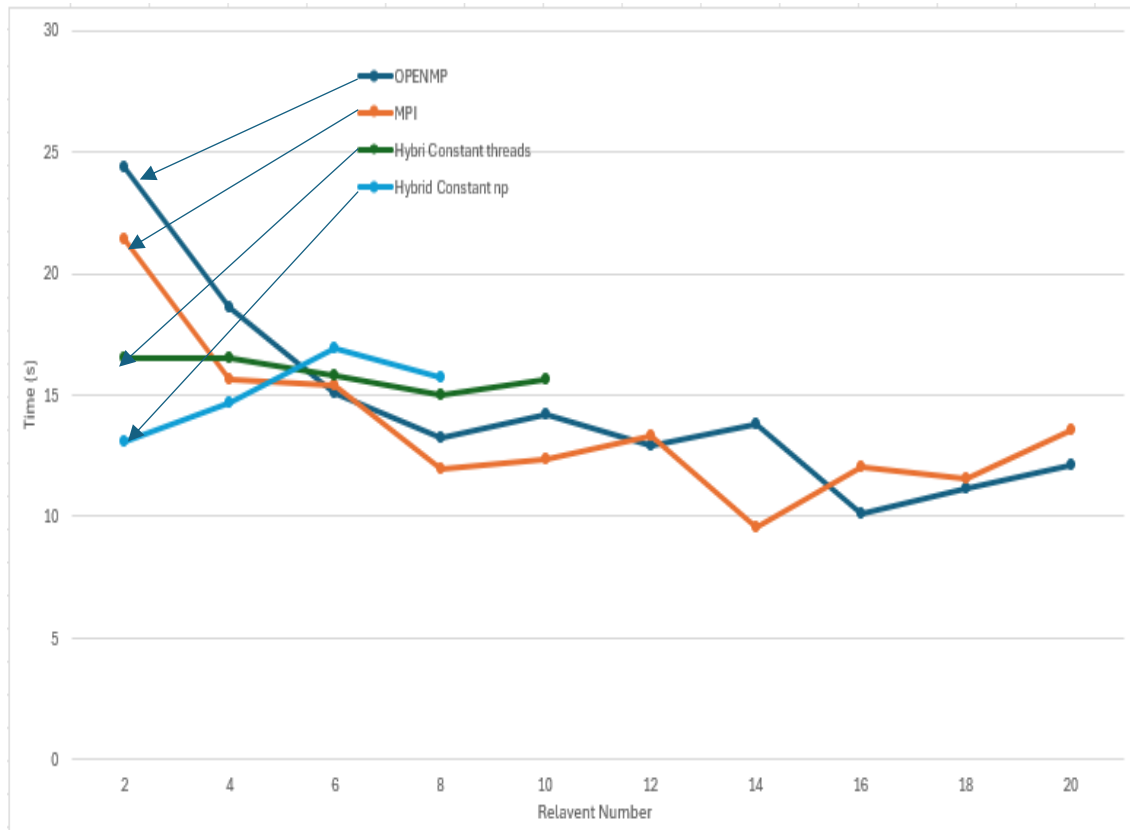
5.1. Within Range

Password cracking is not an approximation task—it gives exact results. For any single password combination tested, the accuracy is either 0% (no match) or 100% (exact match). When considering all possible combinations within a specified range, the program guarantees 100% accuracy: if the password exists in the defined range, it will be found without fail. This is because the brute-force method systematically checks every possible combination, leaving no room for uncertainty.

5.2. Out of Range

If the password is not within the user-defined range (e.g., length or character set), the program is still 100% accurate in confirming its absence. It exhaustively verifies all possible combinations within the given constraints and conclusively determines that the password does not exist in that range. This absolute certainty is a key strength of brute-force methods—there are no false positives or missed matches when the search space is correctly defined.

6. Summery



The graph provides a comparative summary of password cracking performance across different parallel computing methods. The vertical axis represents time in seconds, while the horizontal axis shows relevant test numbers (14, 16, 18, 20). OpenMP (blue) demonstrates consistent performance, while MPI (orange) shows significant speed improvements as processing nodes increase. The hybrid approaches - both constant threads (green) and constant processes (red) - offer intermediate performance between pure OpenMP and MPI implementations. Notably, MPI achieves the lowest times (fastest performance) at higher node counts, particularly at position 20, while hybrid methods balance between thread-based and distributed computing advantages.

7. Conclusion

This project successfully demonstrated the effectiveness of parallel computing techniques for brute-force password cracking. The implementation compared three approaches—serial, OpenMP (multi-core CPU), and MPI (distributed computing)—to evaluate their performance in cracking SHA-256 hashed passwords.

Experimental results showed that OpenMP significantly reduced cracking time compared to the serial version by efficiently utilizing multiple CPU cores. The MPI implementation further improved performance by distributing the workload across multiple machines, proving particularly effective for large-scale password searches. The hybrid approach (OpenMP + MPI) balanced both shared and distributed computing, optimizing resource usage.

The findings highlight how parallel processing accelerates computationally intensive tasks like password cracking. While OpenMP excels on single multi-core systems, MPI extends capabilities to clustered environments, making the solution scalable. This project confirms that proper parallelization can drastically reduce brute-force attack times, complex passwords to resist such methods.

8. References

GitHub Repository

https://github.com/Mahesh-Wijerathna/HPC/tree/main/project_v4

SHA256 converting tool

<https://emn178.github.io/online-tools/sha256.html>